

EE3305/ME3243 Robotic System Design

Part 2 Project

Team 14

Members:

Name	Matriculation Number
Neo Siao Bee, Dawn	[redacted]
Rubesh Suresh	[redacted]
Asahi Sugano	[redacted]

Table of Contents

1. Introduction.....2
2. Controller Node.....3
2.1 Implementation Overview..... 3
2.2 Limitations & Insights.....4
3. Planner Node.....6
3.1 Implementation Overview..... 6
3.2 Limitations & Insights.....7
3.3 Improvements & Testing..... 8
4. System Integration..... 10
4.1 Limitations & Insights.....10

1. Introduction

The field of autonomous navigation has rapidly advanced in recent years, driven by the integration of sophisticated path planning algorithms like A* [1]. In robotic navigation, the A* algorithm has emerged as one of the foundational techniques, widely recognised for its ability to find optimal paths in grid-based environments efficiently [2]. Recent advancements in A*-based algorithms have focused on enhancing its flexibility and robustness through adaptive techniques, inflation zones, and costmap adjustments, allowing robots to navigate safely, even in dynamic, obstacle-dense environments [3].

The real-world applications of A* path planning are extensive, spanning across industries like logistics, autonomous vehicles, and healthcare. For example, in the logistics industry, A* enables warehouse robots to manoeuvre between storage areas efficiently while avoiding obstructions, optimising operations in fulfilment centres [4]. Meanwhile, in autonomous driving, A* variants assist vehicles in navigating roads, finding parking spaces, and avoiding collisions in congested areas [5]. In the healthcare industry, A* guides robotic assistants in hospitals to transport medical supplies autonomously, aiding in time-sensitive deliveries within complex hospital layouts [6].

In this project, we implemented A* path planning within a simulated ROS2 and Gazebo environment with a hands-on approach, to understand and optimise the algorithm in a controlled setting. By addressing the challenges of obstacle avoidance, safe path selection, and adaptive neighbour expansion, this implementation not only aligns with the current state of the art, but also demonstrates the foundational techniques used in real-world autonomous systems, emphasising the practical and scalable nature of A* in solving modern navigation problems.

2. Controller Node

2.1 Implementation Overview

Path and Pose Initialization and Validation
One of the foundational aspects of this controller is ensuring that both path and pose data are valid before any control calculations. This pattern emphasises error-checking and robustness in dynamic environments where data might occasionally be delayed or unavailable. Therefore, data validation is performed as an initial step to prevent processing errors.

```
// Check if robot's pose is available
if (std::isnan(rbt_x) || std::isnan(rbt_y) || std::isnan(rbt_h))
{
    RCLCPP_WARN(this->get_logger(), "Robot pose not available.");
    return;
}
```

Lookahead Point Selection
The pure pursuit approach uses a lookahead distance to identify a target point on the path. During development, tuning this distance was found to have a substantial impact on control responsiveness. A larger lookahead distance smooths the robot's motion but may result in wider turns, while a smaller distance enables tighter following but can make the robot's movement jittery. Therefore, selecting an optimal lookahead distance requires **balancing responsiveness with smoothness**, which is dependent on manual tuning.

Constraining Acceleration and Velocity
Constraining linear and angular velocities is essential for stable robot movement. Testing revealed that without these constraints, the robot could overshoot or oscillate, especially during rapid turns (as also elaborated in the limitations below). The use of maximum linear and angular velocities, alongside acceleration limits, reasonably alleviates instability and it is a common approach. For our use cae, these constraints were tuned according to specific robot capabilities, emphasisng the common engineering tenet of theoretical limits being adjusted based on real-world performance.

Modular Callback Design
Separating different functionalities into specific callbacks (e.g., for odometry and path updates) and using timers for the main control loop facilitates organized, scalable code. This design pattern is especially beneficial in complex systems where clarity and

modularity improve debugging and further development. We adopted a modular design to ensure that changes in one part of the code do not inadvertently affect other functionalities, and we can more easily narrow down and isolate any issues that might arise.

The following code shows the use of callbacks for different functionalities, such as handling path updates and odometry updates. The cbTimerMain is also called periodically through a timer.

```
// Initialize topics for path and odometry
void initTopics()
{
    sub_path = this->create_subscription<nav_msgs::msg::Path>(
        "path", rclcpp::SensorDataQoS(),
        std::bind(&Controller::cbPath, this, std::placeholders::_1));
    sub_odom = this->create_subscription<nav_msgs::msg::Odometry>(
        "odom", rclcpp::SensorDataQoS(),
        std::bind(&Controller::cbOdom, this, std::placeholders::_1));
    pub_cmd_vel = this->create_publisher<geometry_msgs::msg::Twist>(
        "cmd_vel", rclcpp::SystemDefaultsQoS());
}
// Timer callback for main control loop
void initTimers()
{
    timer_main = this->create_wall_timer(
        std::chrono::duration<double>(1.0 / frequency),
        std::bind(&Controller::cbTimerMain, this));
}
```

2.2 Limitations & Insights

The controller node is overall responsible for translating the planned path into executable commands for the robot. During the implementation, several limitations were identified and addressed. One of the limitations is the smooth path following and oscillation reduction. The pure pursuit algorithm can cause the robot to oscillate, especially in narrow passages or when following paths with sharp turns. This oscillation leads to inefficient movement and can increase the risk of collisions.

In our `controller.cpp`, we addressed this limitation by dynamically adjusting the lookahead distance based on the robot's speed and the curvature of the path. By decreasing the lookahead distance in areas with tight turns and increasing it in straight paths, we improved the robot's ability to follow the path smoothly.

In the code, the lookahead point is calculated in the cbTimerMain() function:

```
// Compute curvature
double curvature = 0.0;
if (x_r != 0.0 || y_r != 0.0) {
    curvature = 2.0 * y_r / (x_r * x_r + y_r * y_r);
}

// Adjust lookahead distance based on curvature
double adjusted_lookahead_distance = lookahead_distance / (1 + fabs(curvature));
```

This adjustment ensures that the robot responds appropriately to changes in path curvature, reducing oscillations.

Another limitation is velocity and acceleration constraints. Without considering the robot's physical limitations, the controller might generate velocity commands that exceed the robot's capabilities, leading to poor path tracking and potential mechanical issues. To address such limitations, we implemented velocity and acceleration constraints in the cbTimerMain() function of controller.cpp. This ensures that the robot's linear and angular velocities, as well as their rates of change, remain within specified limits:

```
// Apply linear acceleration constraints
double max_lin_vel_change = max_lin_acc * dt;
double lin_vel_diff = desired_lin_vel - prev_lin_vel;
if (std::abs(lin_vel_diff) > max_lin_vel_change) {
    desired_lin_vel = prev_lin_vel + sgn(lin_vel_diff) * max_lin_vel_change;
}

// Apply linear velocity constraints
desired_lin_vel = std::clamp(desired_lin_vel, -max_lin_vel, max_lin_vel);
```

By implementing these constraints, we ensure that the robot operates safely within its physical capabilities.

Lastly, we improved goal achievement detection as the robot may not accurately detect when it has reached the goal, potentially causing unnecessary movement or failure to stop. To tackle this, we implemented a stopping threshold to determine when the robot is close enough to the goal to stop so that the distance to the goal does not have to be exactly zero:

```
// Compute distance to goal
double dx_goal = goal_x - rbt_x;
double dy_goal = goal_y - rbt_y;
double dist_to_goal = sqrt(dx_goal * dx_goal + dy_goal * dy_goal);

if (dist_to_goal < stop_thres) {
    // Stop the robot
    desired_lin_vel = 0.0;
    desired_ang_vel = 0.0;
    // Publish zero velocities
    geometry_msgs::msg::Twist msg;
    msg.linear.x = desired_lin_vel;
    msg.angular.z = desired_ang_vel;
    pub_cmd_vel->publish(msg);

    RCLCPP_INFO(this->get_logger(), "Goal reached.");
    return;
}
```

This implementation ensures the robot stops appropriately upon reaching the goal.

3. Planner Node

3.1 Implementation Overview

The planner node, planner.cpp, is a heuristic-based planner that implements the A* path planning algorithm and leverages octile distance for efficient pathfinding in a grid-based environment.

The run function implements the A* pathfinding algorithm to find the optimal path between the start and goal points on the 2D grid map of the house. The algorithm begins by creating a structure, PlannerNode, to represent each point on the grid, storing

its coordinates, g-cost (movement cost from the start), h-cost (heuristic estimate to the goal), and a pointer to its parent node, which facilitates path reconstruction.

The function initialises the A* algorithm by calculating the h-cost from the start to the goal using the octile distance, which is ideal for diagonal movement. The starting node is then appended into a priority queue, open_list. This list organises the nodes by their total estimated f-cost (f = g + h), so that nodes closer in proximity to the goal are processed first. A visited matrix is implemented to mark nodes as explored, which prevents redundant calculations.

The main loop of the algorithm repeatedly extracts the node with the lowest f-cost from open_list and checks if it matches the goal. Once the goal has been reached, the algorithm backtracks through the parent pointers to construct the path. However, if the goal has not been reached, the node will be marked as visited and neighbouring nodes which span across eight possible directions, cardinal and diagonal, will be evaluated. Each neighbour's cost is adjusted based on whether its movement type is cardinal or diagonal. If the neighbour is within bounds, not an obstacle, and unvisited, it is added to open_list with updated g-cost, h-cost, and parent values.

Should no path to the goal be found, the algorithm terminates with an output "No path found."

3.2 Limitations & Insights

The run function addresses several limitations of A* pathfinding which are elaborated on below.

One limitation in A* pathfinding is the potential for excessive memory use, especially in large grids. Hence, the implementation of a visited matrix and node cleanup in the open list is aimed to prevent the algorithm from revisiting nodes unnecessarily and hence, optimise memory usage. This was achieved by clearing nodes from the open list and tracking visited nodes effectively to prevent redundant expansions.

```
std::vector<std::vector<bool>>  visited(rows,  std::vector<bool>(cols,
false));
```

Within the main loop, each cell is marked as visited after it has been processed.

```
visited[current->x][current->y] = true;
```

Additionally, instead of retaining every explored node in memory, nodes that no longer contribute to reaching the goal were deleted. This improvement greatly reduces the algorithm's memory footprint, which is an important consideration for real-time applications in constrained environments.

```
while (!open_list.empty()) {
      delete open_list.top();
      open_list.pop();
}
```

As A* pathfinding often produces jagged paths in grid-based systems, an important enhancement to consider was path smoothness. By structuring the PlannerNode with a parent pointer for each node, we are able to trace back from the goal node to the start node. By following the parent pointers, the path gets reconstructed in reverse order, from the goal to the start. This method enables us to capture the complete path without revisiting the entire search space, making it efficient for further manipulation.

The foundation for enhancing path smoothness is established in the following section:

```
PlannerNode* node = current;
while (node) {
      path.push_back(node->x);
      path.push_back(node->y);
      node = node->parent;
}
```

3.3 Improvements & Testing

One improvement made involves refining the neighbour selection process by allowing for diagonal movement and weighting it appropriately. Cardinal moves are assigned a cost of 1, while diagonal moves are weighted with a cost of $\sqrt{2}$, which equals to approximately 1.414. In standard grid-based A* algorithms, diagonal movement is often not allowed. However, we incorporated an octile-distance adjustment to enable smooth diagonal transitions and efficiency in open spaces. In addition to standard cardinal movements, enabling diagonal movements allows for potentially shorter paths to be found, instead of being restricted to a rigid grid pattern.

```
std::vector<std::pair<int, int>> directions = {{-1, -1}, {-1, 1}, {1,
-1}, {1, 1}, {-1, 0}, {1, 0}, {0, -1}, {0, 1}};
```

During extensive testing, edge cases such as unreachable goals and boundary constraints were encountered. The algorithm includes boundary checks within the neighbour exploration phase, which ensures that nodes do not extend beyond the grid limits.

```
if (nx >= 0 && nx < rows && ny >= 0 && ny < cols && !visited[nx][ny]
&& map[nx * cols + ny] == 0) {
```

Moreover, the implementation checks for scenarios whereby no viable path exists, by allowing for early termination if the algorithm exhausts all feasible nodes without reaching the goal. This design consideration prevents wasted computational cycles and provides immediate feedback to the user, in turn enhancing the algorithm's robustness and usability in real-time systems.

```
std::cerr << "No path found." << std::endl;
return {};
```

4. System Integration

Integrating the planner and controller nodes into a cohesive system is essential for robot navigation as the robot has to make a plan and execute it accordingly. Effective system integration addresses some limitations that arise from the interaction between planning and controlling components.

4.1 Limitations & Insights

One of the limitations of our systems is that the controller might act on outdated paths if there is a delay or mismatch in updates. This occurs when the computing path plan takes longer than expected or not received for some reason. To address this problem, it is important to use ROS topics with appropriate quality of service settings to ensure reliable communication. In controller.cpp, the path topic is subscribed using rclcpp::SensorDataQos() as followed:

```
sub_path = this->create_subscription<nav_msgs::msg::Path>(
    "path", rclcpp::SensorDataQoS(),
    std::bind(&Controller::cbPath, this, std::placeholders::_1));
```

Such quality of service ensures that the controller receives the latest path updates promptly, reducing a delay or mismatch.

Another limitation is the potential misconfiguration in Gazebo, which can prevent it from launching correctly through a launch file. This misconfiguration often results in Gazebo failing to load models properly. This issue commonly occurs when testing a launch file, as environment variables are sometimes overridden, causing conflicts that disrupt Gazebo's ability to function as expected. To resolve this issue, we used `. /usr/share/gazebo/setup.sh` to reset environment variables for Gazebo. Running this script ensures that Gazebo's environment variables are correctly set by isolating them from other software, which is crucial for Gazebo to work properly, especially in integrated simulation environments like robot navigation.

A critical limitation during testing is that unhandled errors in one node can disrupt the entire system, making it challenging to identify the root cause of the error. This significantly slows down the debugging process, as it becomes difficult to pinpoint the error source when the integrated system crashes. To streamline debugging, we implemented error checking and logging mechanisms. For instance, in controller.cpp, we added a check to handle cases where the path is empty:

```
if (plan_flat.empty()) {
    RCLCPP_WARN(this->get_logger(), "Path is empty.");
    return;
}
```

By constantly monitoring controller node status, we improved the integrated system's robustness, making it easier to detect and address issues. Once debugging was completed, these error-checking codes were removed from the final product to maintain clean and concise code.

References

[1] M. Reda, A. Onsy, A. Y. Haikal and A. Ghanbari, “Path planning algorithms in the autonomous driving system: A comprehensive review,” Robotics and Autonomous Systems, vol. 174, 2024, doi: 10.1016/j.robot.2024.104630.

[2] T.-V. Dang and D.-S. Nguyen, “Optimal Navigation Based on Improved A* Algorithm for Mobile Robot,” Sep. 2023, doi: 10.1007/978-981-99-4725-6_68.

[3] R. Ribeiro and P. Moreno, “Socially reactive navigation models for mobile robots in dynamic environments,” Instituto Superior Tecnico, Lisboa, Portugal, Oct. 2023.

[4] A. Olukunle, S. Enoch, U. Uchenna and A. Akoh, “AI-driven warehouse automation: A comprehensive review of systems,” GSC Advanced Research and Reviews, vol. 18, pp. 272-282, Feb. 2024, doi: 10.30574/gscarr.2024.18.2.0063.

[5] Y. Zhang, A. Carballo, H. Yang and K. Takeda, “Perception and sensing for autonomous vehicles under adverse weather conditions: A survey,” ISPRS Journal of Photogrammetry and Remote Sensing, vol. 196, pp. 146-177, Feb. 2023, doi: 10.1016/j.isprsjprs.2022.12.021.

[6] A. A. Morgan, J. Abdi, M. A. Q. Syed, G. E. Kohen, P. Barlow and M. P. Vizcaychipi, “Robots in Healthcare: a Scoping Review,” Curr Robot Rep, vol. 3, no. 4, pp. 271-280, Oct. 2022, doi: 10.1007/s43154-022-00095-4.